

CS840a
Learning and Computer Vision
Prof. Olga Veksler

Lecture 8

SVM

Some pictures from C. Burges

SVM

- Said to start in 1979 with Vladimir Vapnik's paper
- Major developments throughout 1990's
- Elegant theory
 - Has good generalization properties
- Have been applied to diverse problems very successfully in the last 15-20 years

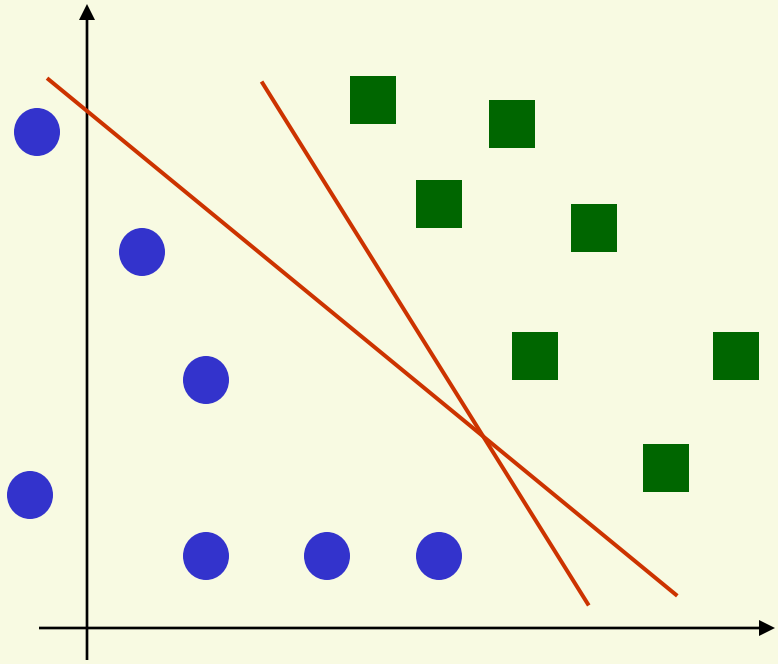


Linear Discriminant Functions

$$g(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + w_0$$

$$g(\mathbf{x}) > 0 \Rightarrow \mathbf{x} \in \text{class 1}$$

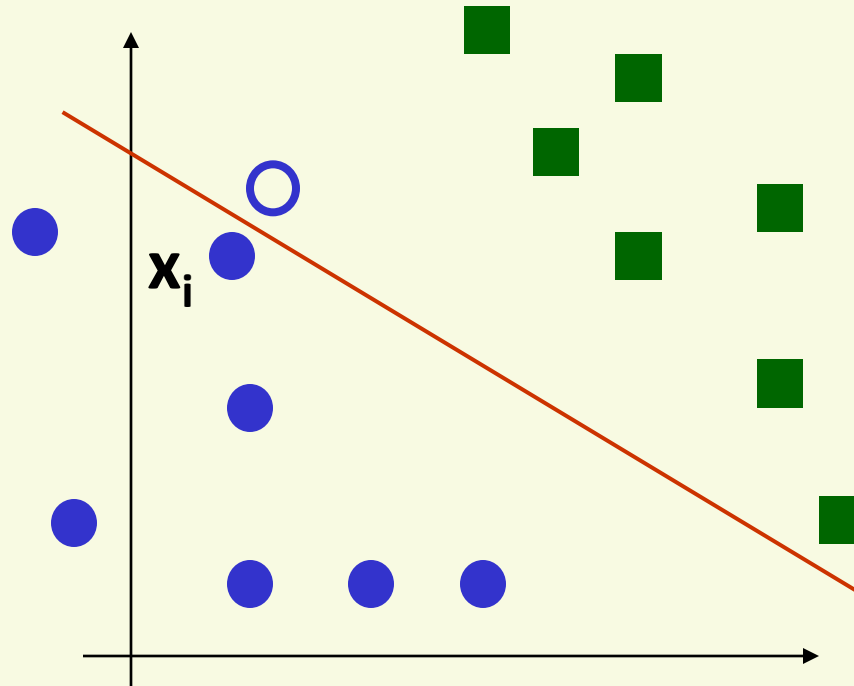
$$g(\mathbf{x}) < 0 \Rightarrow \mathbf{x} \in \text{class 2}$$



- which separating hyperplane should we choose?

Margin Intuition

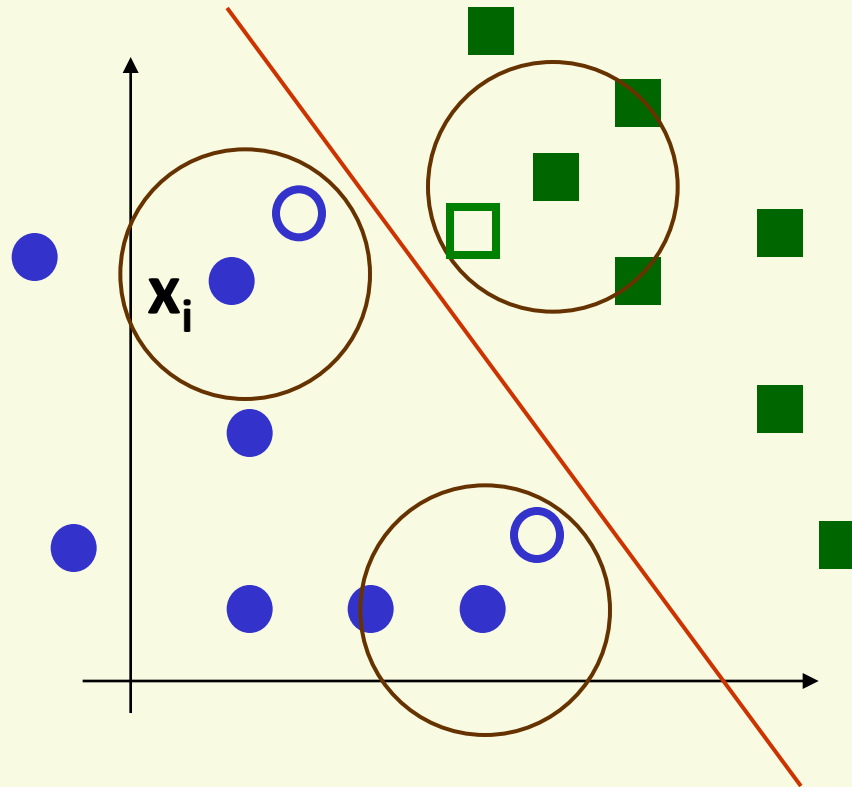
- Training data is just a subset of of all possible data
- Suppose hyperplane is close to sample $\mathbf{x}_i$
- If sample is close to sample $\mathbf{x}_i$, it is likely to be on the wrong side



- Poor generalization

Margin Intuition

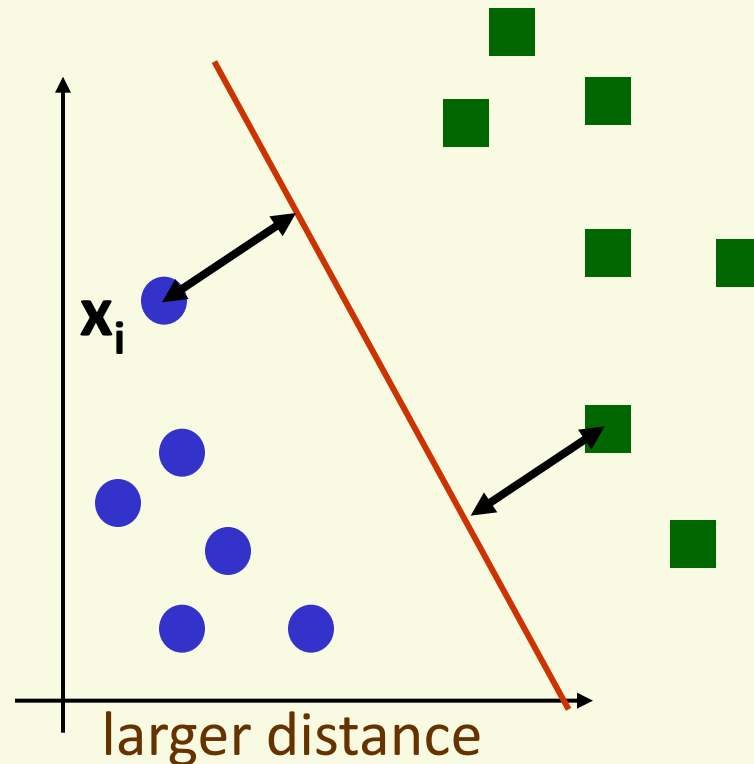
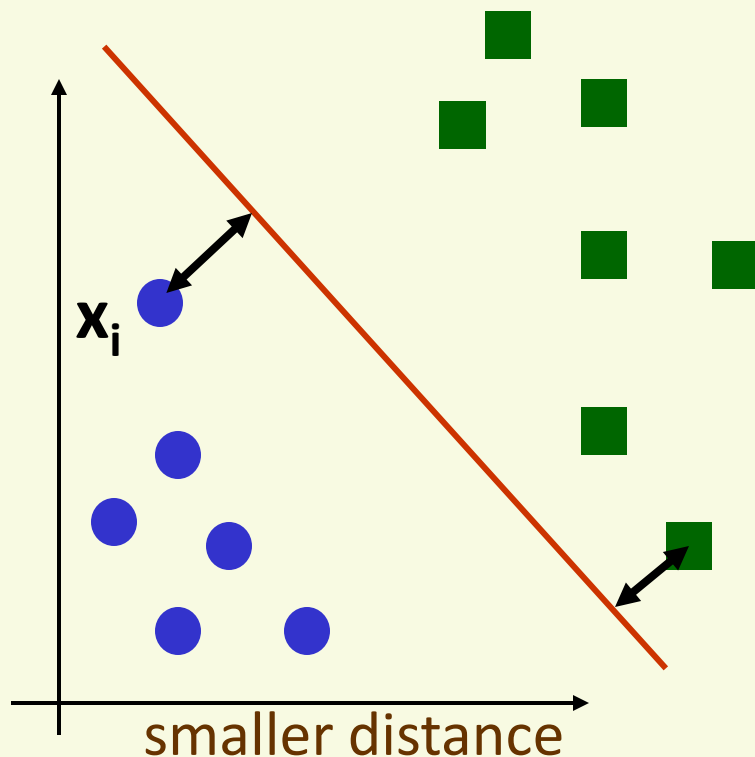
- Hyperplane as far as possible from any sample



- More likely that new samples close to old samples classified correctly
- Good generalization

SVM

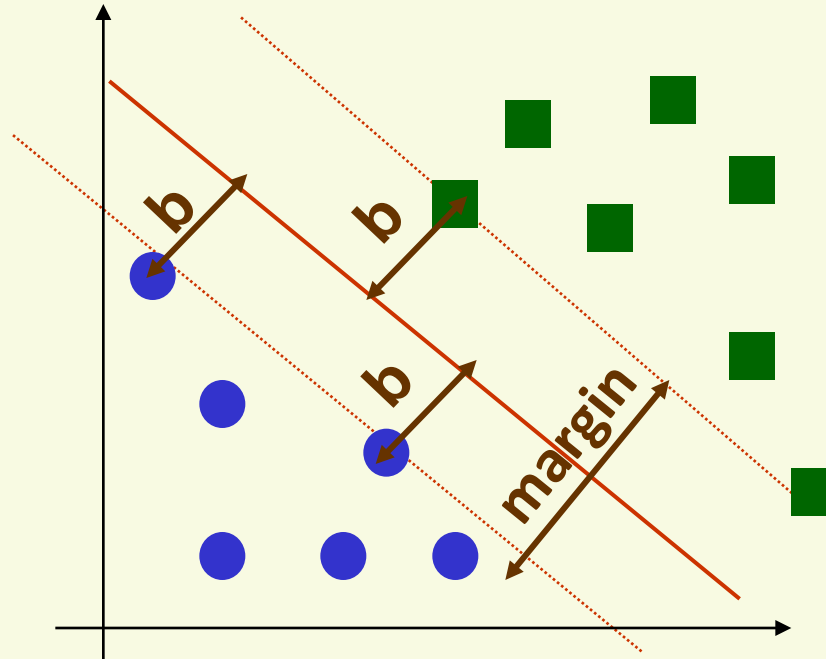
- Idea: maximize distance to the closest example



- For the optimal hyperplane
 - distance to the closest negative example = distance to the closest positive example

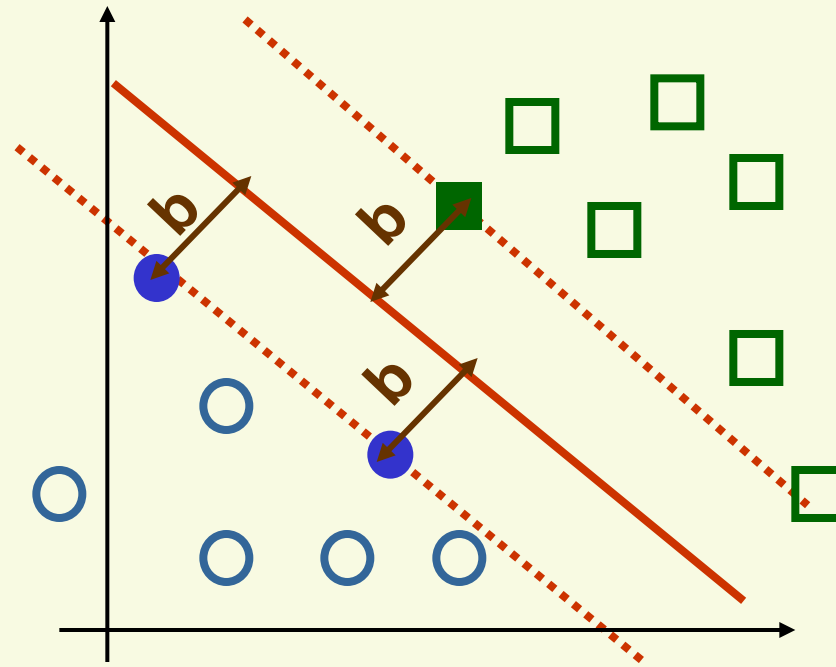
SVM: Linearly Separable Case

- SVM: maximize the *margin*



- *margin* is twice the absolute value of distance b of the closest example to the separating hyperplane
- Better generalization
 - in practice and in theory

SVM: Linearly Separable Case

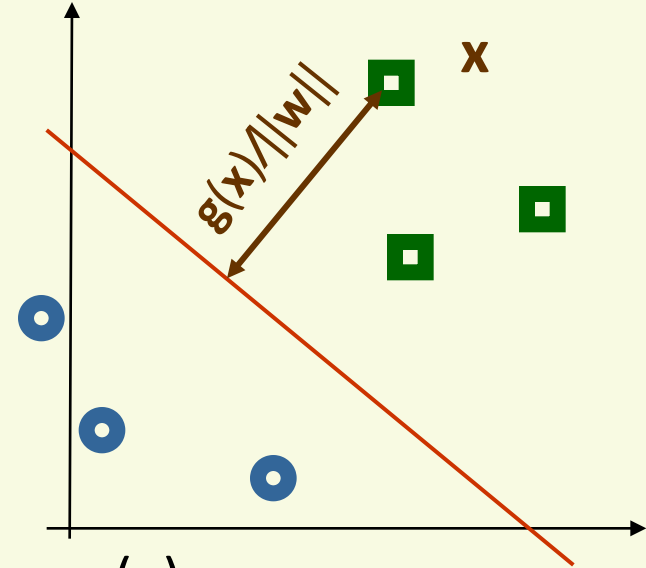


- **Support vectors** are samples closest to separating hyperplane
 - they are the most difficult patterns to classify, intuitively
 - optimal hyperplane is completely defined by support vectors
 - do not know which samples are support vectors beforehand

SVM: Formula for the Margin

- $g(\mathbf{x}) = \mathbf{w}^t \mathbf{x} + \mathbf{w}_0$
- absolute distance between $\mathbf{x}$ and the boundary $g(\mathbf{x}) = 0$

$$\frac{|\mathbf{w}^t \mathbf{x} + \mathbf{w}_0|}{\|\mathbf{w}\|}$$



- distance is unchanged for hyperplane $g_1(\mathbf{x}) = \alpha g(\mathbf{x})$

$$\frac{|\alpha \mathbf{w}^t \mathbf{x} + \alpha \mathbf{w}_0|}{\|\alpha \mathbf{w}\|} = \frac{|\mathbf{w}^t \mathbf{x} + \mathbf{w}_0|}{\|\mathbf{w}\|}$$

- Let $\mathbf{x}_i$ be an example closest to the boundary. Set

$$|\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0| = 1$$

- Now the largest margin hyperplane is unique

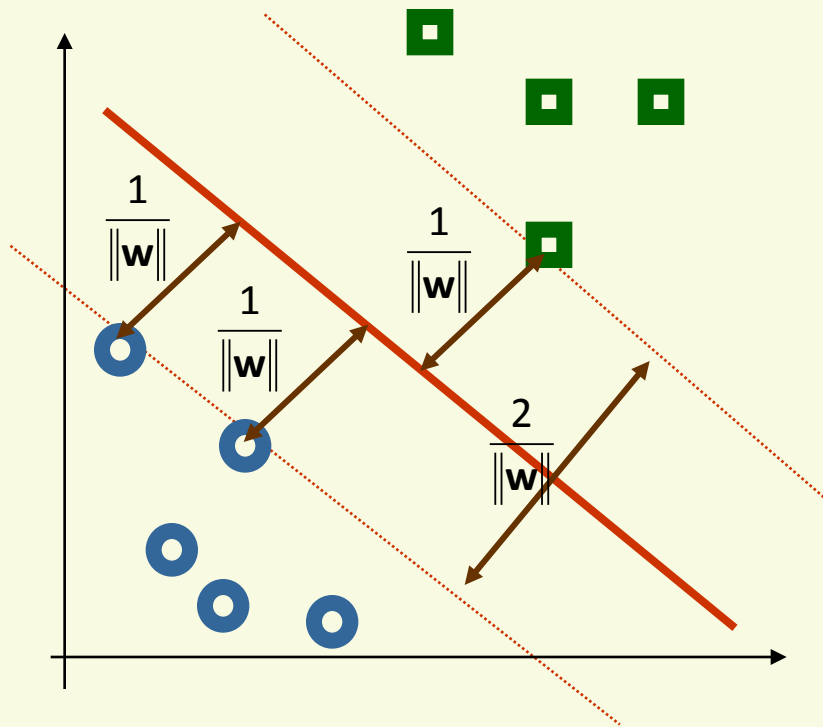
SVM: Formula for the Margin

- For uniqueness, set $|\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0| = 1$ for any example $\mathbf{x}_i$ closest to the boundary
- now distance from closest sample $\mathbf{x}_i$ to $\mathbf{g}(\mathbf{x}) = 0$ is

$$\frac{|\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

- Thus the margin is

$$m = \frac{2}{\|\mathbf{w}\|}$$



SVM: Optimal Hyperplane

- Maximize margin $\mathbf{m} = \frac{2}{\|\mathbf{w}\|}$
- subject to constraints
$$\begin{cases} \mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0 \geq 1 & \text{if } \mathbf{x}_i \text{ is positive example} \\ \mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0 \leq -1 & \text{if } \mathbf{x}_i \text{ is negative example} \end{cases}$$

- Let
$$\begin{cases} \mathbf{z}_i = 1 & \text{if } \mathbf{x}_i \text{ is positive example} \\ \mathbf{z}_i = -1 & \text{if } \mathbf{x}_i \text{ is negative example} \end{cases}$$

- Convert our problem to

$$\begin{aligned} &\text{minimize} && J(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 \\ &\text{constrained to} && \mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 \quad \forall i \end{aligned}$$

- $J(\mathbf{w})$ is a convex function, thus it has a single global minimum

SVM: Optimal Hyperplane

- Use Kuhn-Tucker theorem to convert our problem to:

$$\begin{aligned} &\text{maximize} && \mathbf{L}_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j \\ &\text{constrained to} && \alpha_i \geq 0 \quad \forall i \quad \text{and} \quad \sum_{i=1}^n \alpha_i \mathbf{z}_i = 0 \end{aligned}$$

- $\alpha = \{\alpha_1, \dots, \alpha_n\}$ are new variables, one for each sample
- Rewrite $\mathbf{L}_D(\alpha)$ using n by n matrix $\mathbf{H}$:

$$\mathbf{L}_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix}^t \mathbf{H} \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix}$$

- where the value in the i th row and j th column of $\mathbf{H}$ is

$$\mathbf{H}_{ij} = \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j$$

SVM: Optimal Hyperplane

- Use Kuhn-Tucker theorem to convert our problem to:

$$\begin{array}{ll} \text{maximize} & \mathbf{L}_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j \\ \text{constrained to} & \alpha_i \geq 0 \quad \forall i \quad \text{and} \quad \sum_{i=1}^n \alpha_i \mathbf{z}_i = 0 \end{array}$$

- $\alpha = \{\alpha_1, \dots, \alpha_n\}$ are new variables, one for each sample
- $\mathbf{L}_D(\alpha)$ can be optimized by quadratic programming
- $\mathbf{L}_D(\alpha)$ formulated in terms of α
 - depends on $\mathbf{w}$ and $\mathbf{w}_0$

SVM: Optimal Hyperplane

- After finding the optimal $\alpha = \{\alpha_1, \dots, \alpha_n\}$
 - for every sample i , one of the following must hold
 - $\alpha_i = 0$ (sample i is not a support vector)
 - $\alpha_i \neq 0$ and $\mathbf{z}_i(\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0 - 1) = 0$ (sample i is support vector)
- compute $\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{z}_i \mathbf{x}_i$
- solve for $\mathbf{w}_0$ using any $\alpha_i > 0$ and $\alpha_i [\mathbf{z}_i(\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) - 1] = 0$
$$\mathbf{w}_0 = \frac{1}{\mathbf{z}_i} - \mathbf{w}^t \mathbf{x}_i$$
- Final discriminant function:

$$\mathbf{g}(\mathbf{x}) = \left(\sum_{\mathbf{x}_i \in \mathbf{S}} \alpha_i \mathbf{z}_i \mathbf{x}_i \right)^t \mathbf{x} + \mathbf{w}_0$$

- where $\mathbf{S}$ is the set of support vectors

$$\mathbf{S} = \{\mathbf{x}_i \mid \alpha_i \neq 0\}$$

SVM: Optimal Hyperplane

maximize

$$L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j$$

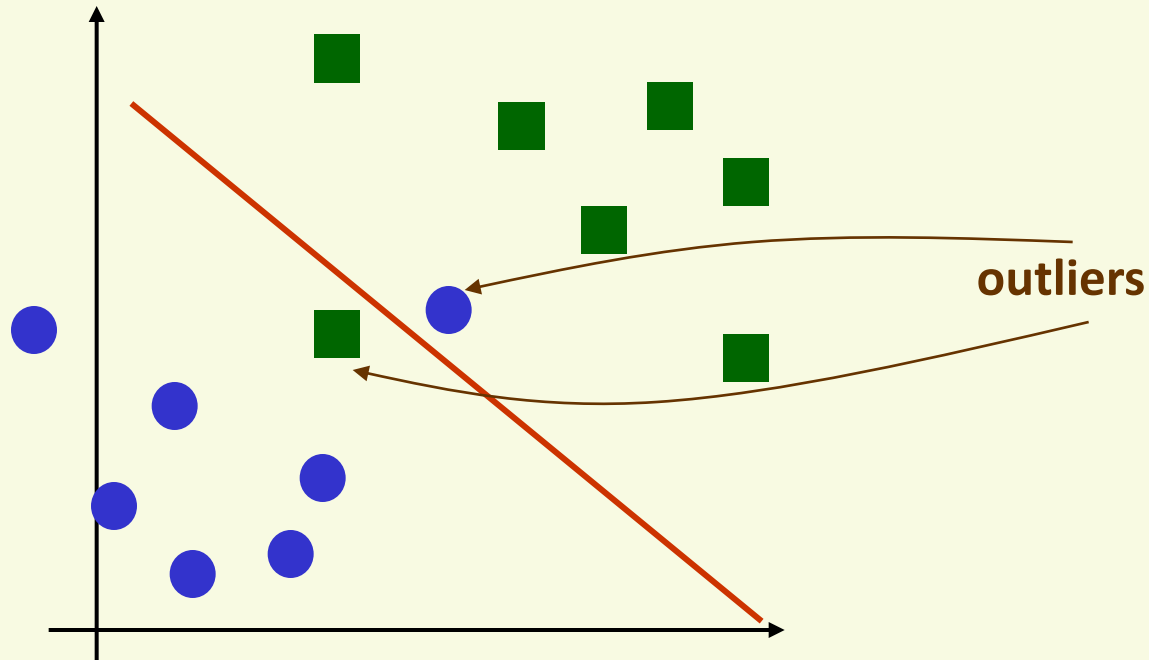
constrained to

$$\alpha_i \geq 0 \quad \forall i \quad \text{and} \quad \sum_{i=1}^n \alpha_i \mathbf{z}_i = 0$$

- $L_D(\alpha)$ depends on the number of samples, not on dimension of samples
- samples appear only through the dot products $\mathbf{x}_i^t \mathbf{x}_j$
- Will become important when looking for a ***nonlinear*** discriminant function

SVM: Non Separable Case

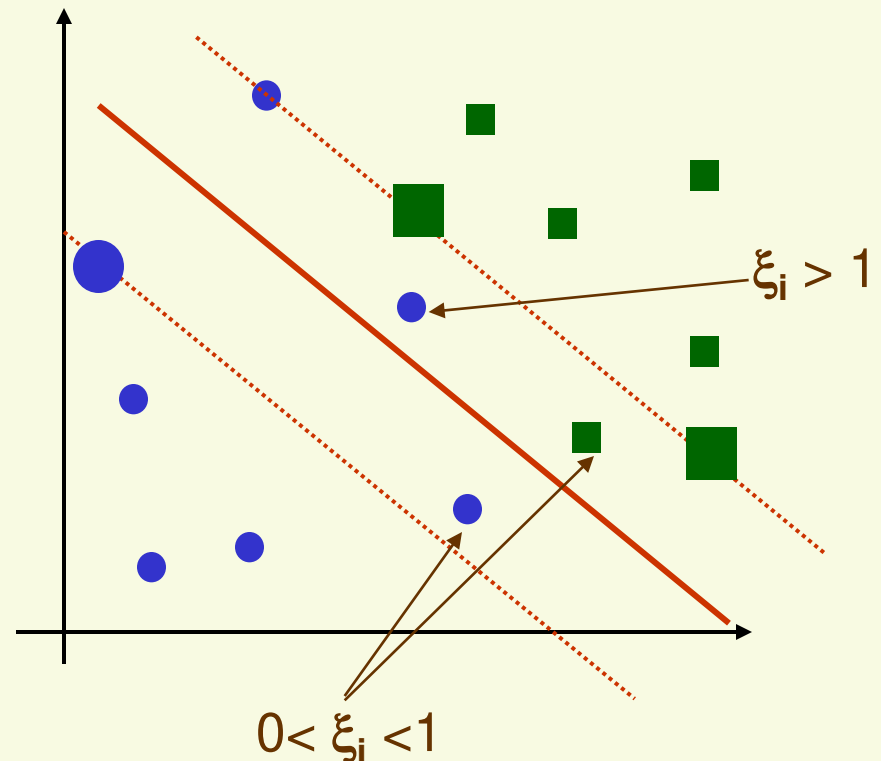
- Linear classifier still be appropriate when data is not linearly separable, but almost linearly separable



- Can adapt SVM to almost linearly separable case

SVM: Non Separable Case

- Introduce non-negative *slack* variables $\xi_1, \dots, \xi_n$
 - one for each sample
- Change constraints from $\mathbf{z}_i(\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 \quad \forall i$ to
$$\mathbf{z}_i(\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 - \xi_i \quad \forall i$$
- ξ_i measures deviation from the ideal position for sample $\mathbf{x}_i$
 - $\xi_i > 1$: $\mathbf{x}_i$ is on the wrong side of the hyperplane
 - $0 < \xi_i < 1$: $\mathbf{x}_i$ is on the right side of the hyperplane but within the region of maximum margin



SVM: Non Separable Case

- Wish to minimize

$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n I(\xi_i > 0)$$

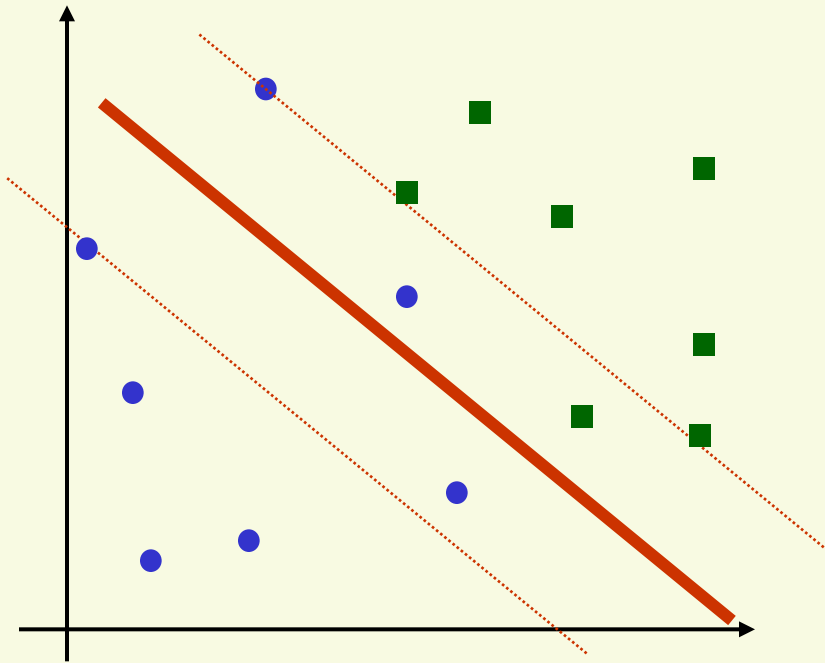
of samples
not in ideal location

- where $I(\xi_i > 0) = \begin{cases} 1 & \text{if } \xi_i > 0 \\ 0 & \text{if } \xi_i \leq 0 \end{cases}$
- constrained to $\mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 - \xi_i$ and $\xi_i \geq 0 \quad \forall i$
- β measures relative weight of first and second terms
 - if β is small, we allow a lot of samples not in ideal position
 - if β is large, we allow very few samples not in ideal position
 - choosing β appropriately is important

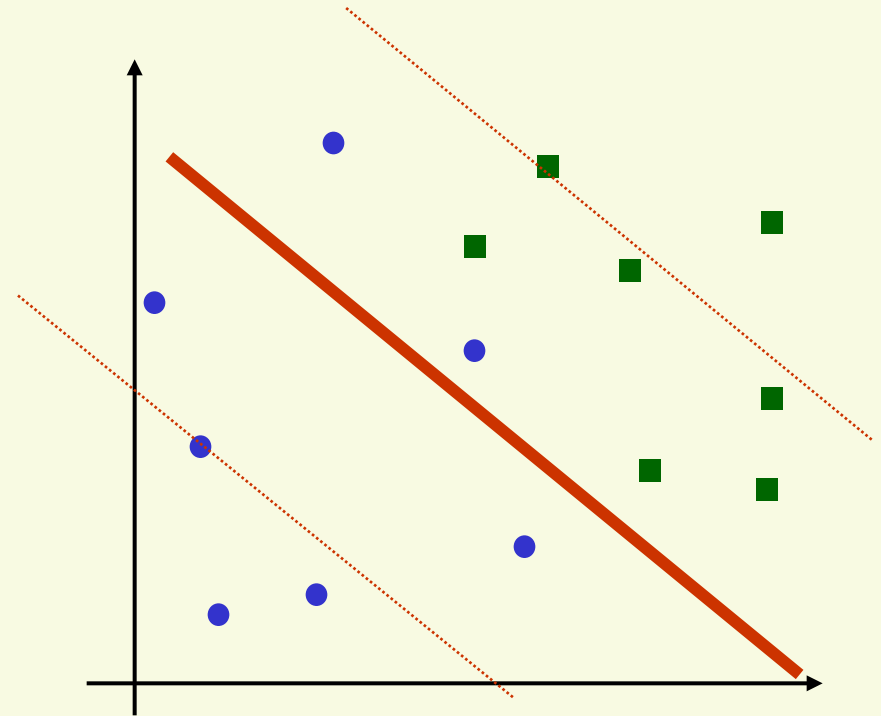
SVM: Non Separable Case

$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n I(\xi_i > 0)$$

of samples
not in ideal location



large β , few samples not in
ideal position



small β , many samples not in
ideal position

SVM: Non Separable Case

- Minimization problem is NP-hard due to discontinuity of $I(\xi_i)$

$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n I(\xi_i > 0)$$

of samples
not in ideal location

- where $I(\xi_i > 0) = \begin{cases} 1 & \text{if } \xi_i > 0 \\ 0 & \text{if } \xi_i \leq 0 \end{cases}$
- constrained to $\mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 - \xi_i$ and $\xi_i \geq 0 \quad \forall i$

SVM: Non Separable Case

- Instead we minimize

$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n \xi_i$$

a measure of
of misclassified
examples

- constrained to
$$\begin{cases} \mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 - \xi_i & \forall i \\ \xi_i \geq 0 & \forall i \end{cases}$$

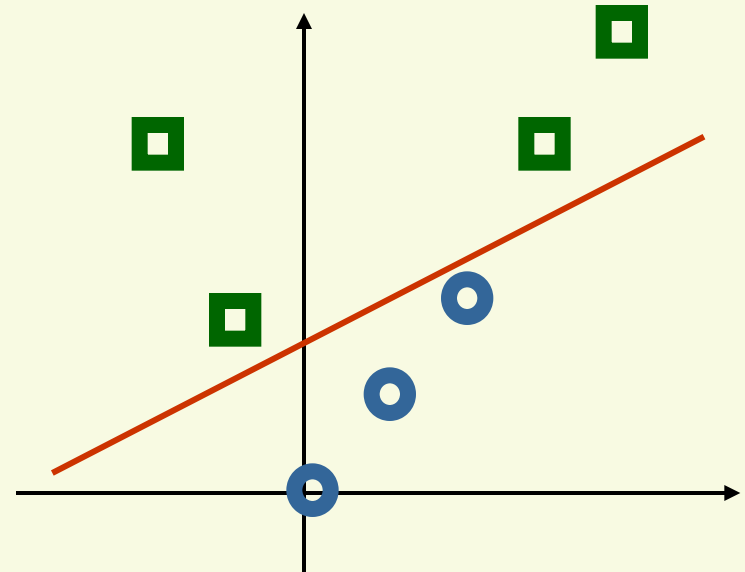
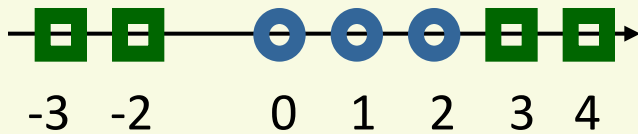
- Use Kuhn-Tucker theorem to converted to

$$\begin{aligned} \text{maximize} \quad & L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j \\ \text{constrained to} \quad & 0 \leq \alpha_i \leq \beta \quad \forall i \quad \text{and} \quad \sum_{i=1}^n \alpha_i \mathbf{z}_i = 0 \end{aligned}$$

- find $\mathbf{w}$ using
$$\mathbf{w} = \sum_{i=1}^n \alpha_i \mathbf{z}_i \mathbf{x}_i$$
- solve for $\mathbf{w}_0$ using any $0 < \alpha_i < \beta$ and $\alpha_i [\mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) - 1] = 0$

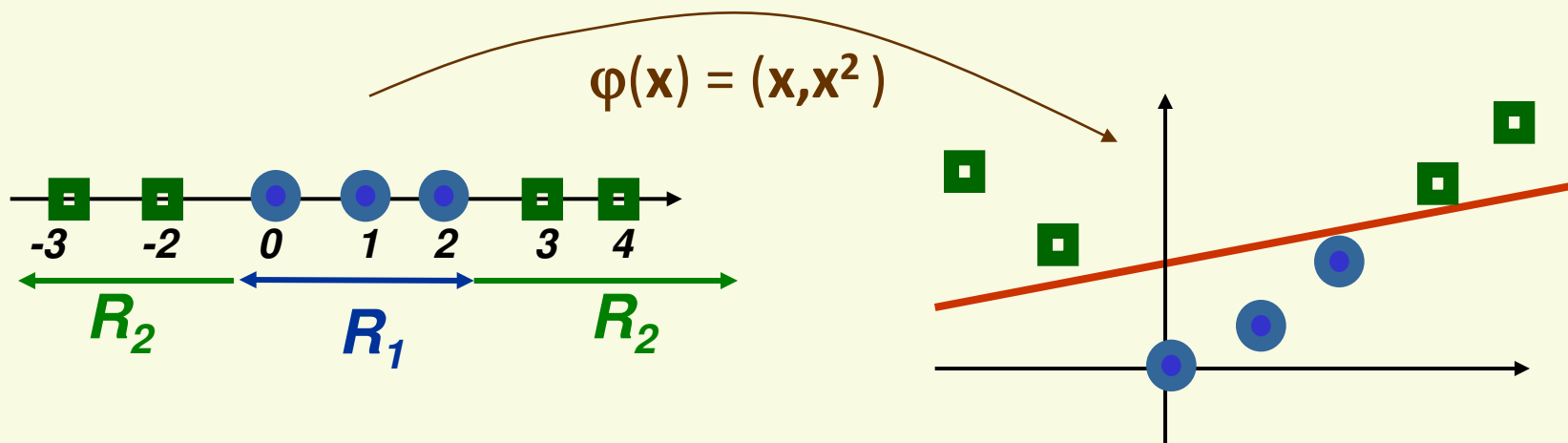
Non Linear Mapping

- Cover's theorem:
 - “*pattern-classification problem cast in a high dimensional space non-linearly is more likely to be linearly separable than in a low-dimensional space*”
- Not linearly separable in 1D
- Lift to 2D space with $\mathbf{h}(\mathbf{x}) = (\mathbf{x}, \mathbf{x}^2)$



Non Linear Mapping

- To solve a non linear problem with a linear classifier
 1. Project data $\mathbf{x}$ to high dimension using function $\phi(\mathbf{x})$
 2. Find a linear discriminant function for transformed data $\phi(\mathbf{x})$
 3. Final nonlinear discriminant function is $\mathbf{g}(\mathbf{x}) = \mathbf{w}^t \phi(\mathbf{x}) + \mathbf{w}_0$

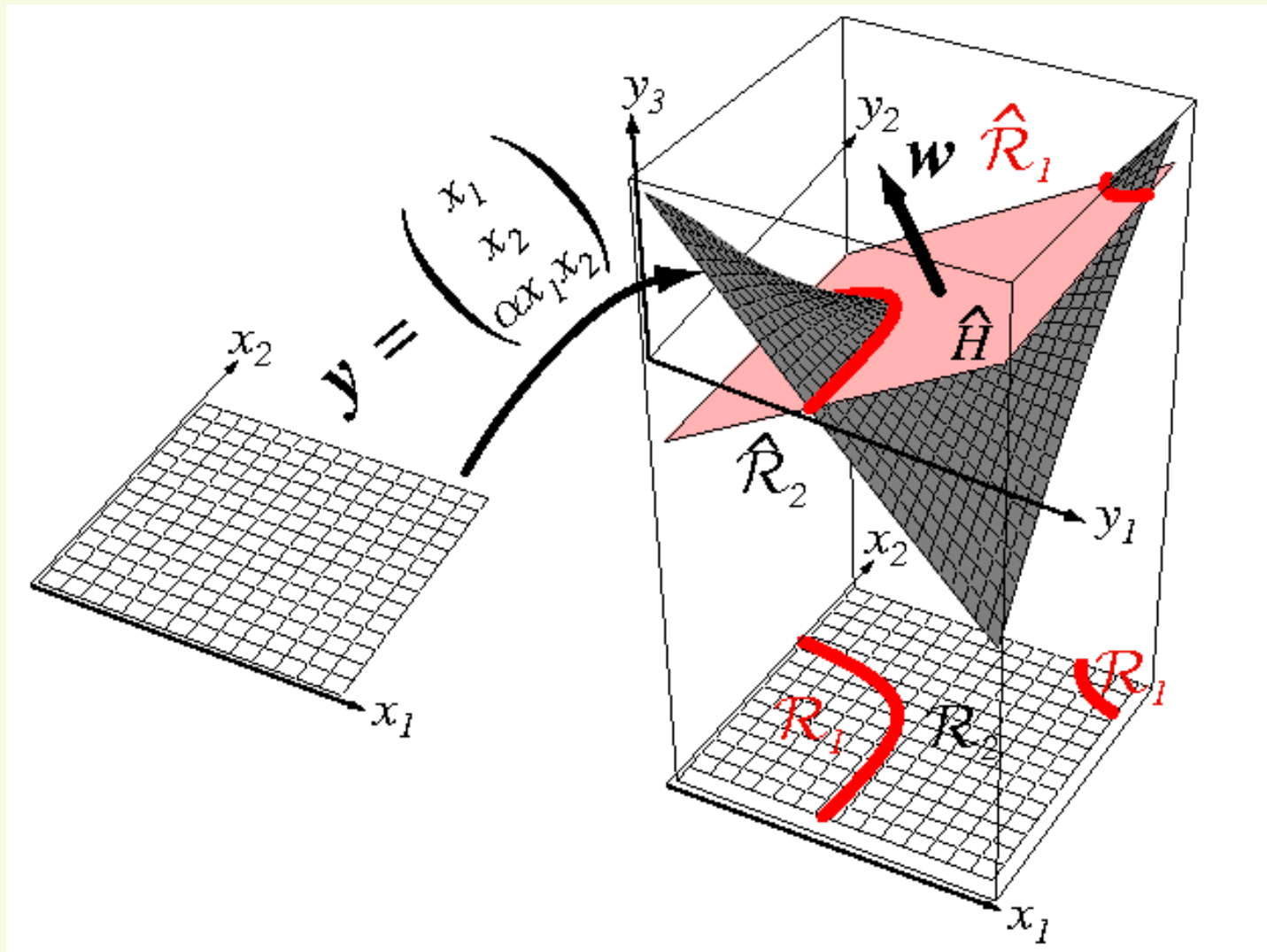


- In 2D, discriminant function is linear

$$\mathbf{g}\left(\begin{bmatrix} \mathbf{x}^{(1)} \\ \mathbf{x}^{(2)} \end{bmatrix}\right) = [\mathbf{w}_1 \quad \mathbf{w}_2] \begin{bmatrix} \mathbf{x}^{(1)} \\ \mathbf{x}^{(2)} \end{bmatrix} + \mathbf{w}_0$$

- In 1D, discriminant function is not linear $\mathbf{g}(\mathbf{x}) = \mathbf{w}_1 \mathbf{x} + \mathbf{w}_2 \mathbf{x}^2 + \mathbf{w}_0$

Non Linear Mapping: Another Example



Non Linear SVM

- Can use any linear classifier after lifting data into a higher dimensional space
- However we will have to deal with the “curse of dimensionality”
 1. poor generalization to test data
 2. computationally expensive
- SVM avoids the “curse of dimensionality” by
 - enforcing largest margin permits good generalization
 - computation in the higher dimensional case is performed only implicitly through the use of *kernel* functions

Non Linear SVM: Kernels

- Recall SVM optimization

$$\text{maximize} \quad L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \mathbf{x}_i^t \mathbf{x}_j$$

- Optimization depends on samples $\mathbf{x}_i$ only through the dot product $\mathbf{x}_i^t \mathbf{x}_j$
- If we lift $\mathbf{x}_i$ to high dimension using $\boldsymbol{\varphi}(\mathbf{x})$, need to compute high dimensional product $\boldsymbol{\varphi}(\mathbf{x}_i)^t \boldsymbol{\varphi}(\mathbf{x}_j)$

$$\text{maximize} \quad L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \underbrace{\boldsymbol{\varphi}(\mathbf{x}_i)^t \boldsymbol{\varphi}(\mathbf{x}_j)}_{\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)}$$

- Idea: find **kernel** function $\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)$ s.t. $\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\varphi}(\mathbf{x}_i)^t \boldsymbol{\varphi}(\mathbf{x}_j)$

Non Linear SVM: Kernels

maximize
$$L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j \underbrace{\varphi(\mathbf{x}_i)^t \varphi(\mathbf{x}_j)}_{\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)}$$

- Kernel trick
 - only need to compute $\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)$ instead of $\varphi(\mathbf{x}_i)^t \varphi(\mathbf{x}_j)$
 - no need to lift data in high dimension explicitly, computation is performed in the original dimension

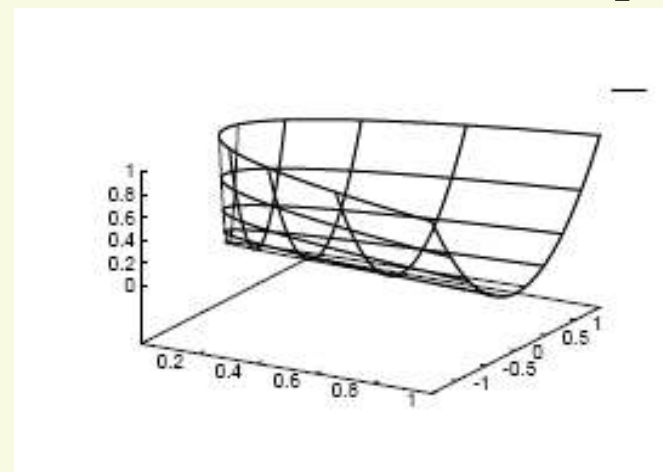
Non Linear SVM: Kernels

- Suppose we have 2 features and $\mathbf{K}(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^t \mathbf{y})^2$
- Which mapping $\boldsymbol{\varphi}(\mathbf{x})$ does it correspond to?

$$\begin{aligned}\mathbf{K}(\mathbf{x}, \mathbf{y}) &= (\mathbf{x}^t \mathbf{y})^2 = \left(\begin{bmatrix} \mathbf{x}^{(1)} & \mathbf{x}^{(2)} \end{bmatrix} \begin{bmatrix} \mathbf{y}^{(1)} \\ \mathbf{y}^{(2)} \end{bmatrix} \right)^2 = (\mathbf{x}^{(1)} \mathbf{y}^{(1)} + \mathbf{x}^{(2)} \mathbf{y}^{(2)})^2 \\ &= (\mathbf{x}^{(1)} \mathbf{y}^{(1)})^2 + 2(\mathbf{x}^{(1)} \mathbf{y}^{(1)})(\mathbf{x}^{(2)} \mathbf{y}^{(2)}) + (\mathbf{x}^{(2)} \mathbf{y}^{(2)})^2 \\ &= \begin{bmatrix} (\mathbf{x}^{(1)})^2 & \sqrt{2} \mathbf{x}^{(1)} \mathbf{x}^{(2)} & (\mathbf{x}^{(2)})^2 \end{bmatrix} \begin{bmatrix} (\mathbf{y}^{(1)})^2 & \sqrt{2} \mathbf{y}^{(1)} \mathbf{y}^{(2)} & (\mathbf{y}^{(2)})^2 \end{bmatrix}^t\end{aligned}$$

- Thus

$$\boldsymbol{\varphi}(\mathbf{x}) = \begin{bmatrix} (\mathbf{x}^{(1)})^2 & \sqrt{2} \mathbf{x}^{(1)} \mathbf{x}^{(2)} & (\mathbf{x}^{(2)})^2 \end{bmatrix}$$



Non Linear SVM: Kernels

- How to choose kernel $K(\mathbf{x}_i, \mathbf{x}_j)$?
 - $K(\mathbf{x}_i, \mathbf{x}_j)$ should correspond to product $\boldsymbol{\varphi}(\mathbf{x}_i)^t \boldsymbol{\varphi}(\mathbf{x}_j)$ in a higher dimensional space
 - Mercer's condition states which kernel function can be expressed as dot product of two vectors
 - Kernel's not satisfying Mercer's condition can be sometimes used, but no geometrical interpretation
- Common choices satisfying Mercer's condition
 - Polynomial kernel $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^t \mathbf{x}_j + 1)^p$
 - Gaussian radial Basis kernel (data is lifted in infinite dimensions)

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$$

Non Linear SVM

- search for separating hyperplane in high dimension

$$\mathbf{w}\varphi(\mathbf{x}) + \mathbf{w}_0 = 0$$

- Choose $\varphi(\mathbf{x})$ so that the first ("0"th) dimension is the augmented dimension with feature value fixed to 1

$$\varphi(\mathbf{x}) = \begin{bmatrix} 1 & \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \mathbf{x}^{(1)}\mathbf{x}^{(2)} \end{bmatrix}^t$$

- Threshold $\mathbf{w}_0$ gets folded into vector $\mathbf{w}$

$$\begin{bmatrix} \mathbf{w}_0 & \mathbf{w} \end{bmatrix} \underbrace{\begin{bmatrix} 1 \\ * \end{bmatrix}}_{\varphi(\mathbf{x})} = 0$$

Non Linear SVM

- Thus seeking hyperplane

$$\mathbf{w}\phi(\mathbf{x}) = 0$$

- Or, equivalently, a hyperplane that goes through the origin in high dimensions
 - removes only one degree of freedom
 - but we introduced many new degrees when lifted the data in high dimension

Non Linear SVM Recipe

- Start with $\mathbf{x}_1, \dots, \mathbf{x}_n$ in original feature space of dimension d
- Choose kernel $K(\mathbf{x}_i, \mathbf{x}_j)$
 - implicitly chooses function $\boldsymbol{\varphi}(\mathbf{x}_i)$ that takes $\mathbf{x}_i$ to a higher dimensional space
 - gives dot product in the high dimensional space
- Find largest margin linear classifier in the higher dimensional space by using quadratic programming package to solve

$$\begin{array}{ll} \text{maximize} & L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{z}_i \mathbf{z}_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{constrained to} & 0 \leq \alpha_i \leq \beta \quad \forall i \quad \text{and} \quad \sum_{i=1}^n \alpha_i \mathbf{z}_i = 0 \end{array}$$

Non Linear SVM Recipe

- Weight vector $\mathbf{w}$ in the high dimensional space

$$\mathbf{w} = \sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \phi(\mathbf{x}_i)$$

- where $\mathcal{S}$ is the set of support vectors

$$\mathcal{S} = \{\mathbf{x}_i \mid \alpha_i \neq 0\}$$

- Linear discriminant function in the high dimensional space

$$\mathbf{g}(\phi(\mathbf{x})) = \mathbf{w}^t \phi(\mathbf{x}) = \left(\sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \phi(\mathbf{x}_i) \right)^t \phi(\mathbf{x})$$

- Non linear discriminant function in the original space:

$$\mathbf{g}(\mathbf{x}) = \left(\sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \phi(\mathbf{x}_i) \right)^t \phi(\mathbf{x}) = \sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \phi^t(\mathbf{x}_i) \phi(\mathbf{x}) = \sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \mathbf{K}(\mathbf{x}_i, \mathbf{x})$$

- Decide class 1 if $\mathbf{g}(\mathbf{x}) > 0$, otherwise decide class 2

Non Linear SVM

- Nonlinear discriminant function

$$\mathbf{g}(\mathbf{x}) = \sum_{\mathbf{x}_i \in \mathcal{S}} \alpha_i \mathbf{z}_i \mathbf{K}(\mathbf{x}_i, \mathbf{x})$$

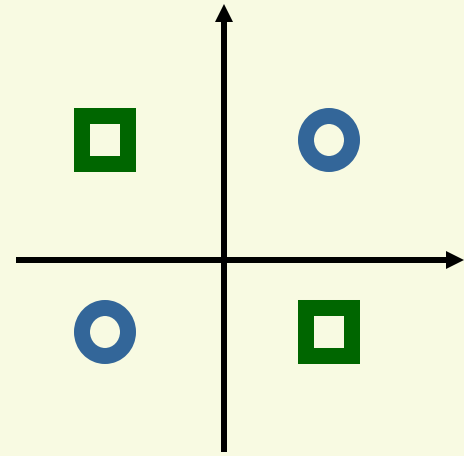
$$\mathbf{g}(\mathbf{x}) = \sum \begin{array}{|c|} \hline \text{weight of support} \\ \text{vector } \mathbf{x}_i \\ \hline \end{array} \begin{array}{|c|} \hline \mp 1 \\ \hline \end{array} \begin{array}{|c|} \hline \text{similarity} \\ \text{between } \mathbf{x} \text{ and} \\ \text{support vector } \mathbf{x}_i \\ \hline \end{array}$$

most important
training samples,
i.e. support vectors

$$\mathbf{K}(\mathbf{x}_i, \mathbf{x}) = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x}_i - \mathbf{x}\|^2\right)$$

SVM Example: XOR Problem

- Class 1: $\mathbf{x}_1 = [1, -1]$, $\mathbf{x}_2 = [-1, 1]$
- Class 2: $\mathbf{x}_3 = [1, 1]$, $\mathbf{x}_4 = [-1, -1]$
- Use polynomial kernel of degree 2



- $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^t \mathbf{x}_j + 1)^2$

- Kernel corresponds to mapping

$$(\mathbf{x}) = \begin{bmatrix} 1 & \sqrt{2}\mathbf{x}^{(1)} & \sqrt{2}\mathbf{x}^{(2)} & \sqrt{2}\mathbf{x}^{(1)}\mathbf{x}^{(2)} & (\mathbf{x}^{(1)})^2 & (\mathbf{x}^{(2)})^2 \end{bmatrix}^t$$

- Need to maximize $L_D(\alpha) = \sum_{i=1}^4 \alpha_i \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \alpha_i \alpha_j z_i z_j (\mathbf{x}_i^t \mathbf{x}_j + 1)^2$
constrained to $0 \leq \alpha_i \quad \forall i$ and $\alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0$

SVM Example: XOR Problem

- Rewrite
$$\mathbf{L}_D(\alpha) = \sum_{i=1}^4 \alpha_i - \frac{1}{2} \alpha^t \mathbf{H} \alpha$$
 - where $\alpha = [\alpha_1 \quad \alpha_2 \quad \alpha_3 \quad \alpha_4]^t$ and $\mathbf{H} = \begin{bmatrix} 9 & 1 & -1 & -1 \\ 1 & 9 & -1 & -1 \\ -1 & -1 & 9 & 1 \\ -1 & -1 & 1 & 9 \end{bmatrix}$

- Take derivative with respect to α and set it to $\mathbf{0}$

$$\frac{d}{d\alpha} \mathbf{L}_D(\alpha) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 9 & 1 & -1 & -1 \\ 1 & 9 & -1 & -1 \\ -1 & -1 & 9 & 1 \\ -1 & -1 & 1 & 9 \end{bmatrix} \alpha = 0$$

- Solution to the above is $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = \mathbf{0.25}$
 - satisfies the constraints $\forall i, 0 \leq \alpha_i$ and $\alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0$
 - all samples are support vectors

SVM Example: XOR Problem

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 & \sqrt{2}\mathbf{x}^{(1)} & \sqrt{2}\mathbf{x}^{(2)} & \sqrt{2}\mathbf{x}^{(1)}\mathbf{x}^{(2)} & (\mathbf{x}^{(1)})^2 & (\mathbf{x}^{(2)})^2 \end{bmatrix}^t$$

- Weight vector $\mathbf{w}$ is:

$$\begin{aligned} \mathbf{w} &= \sum_{i=1}^4 \alpha_i \mathbf{z}_i \phi(\mathbf{x}_i) = 0.25(\phi(\mathbf{x}_1) + \phi(\mathbf{x}_2) - \phi(\mathbf{x}_3) - \phi(\mathbf{x}_4)) \\ &= \begin{bmatrix} 0 & 0 & 0 & \sqrt{2} & 0 & 0 \end{bmatrix} \end{aligned}$$

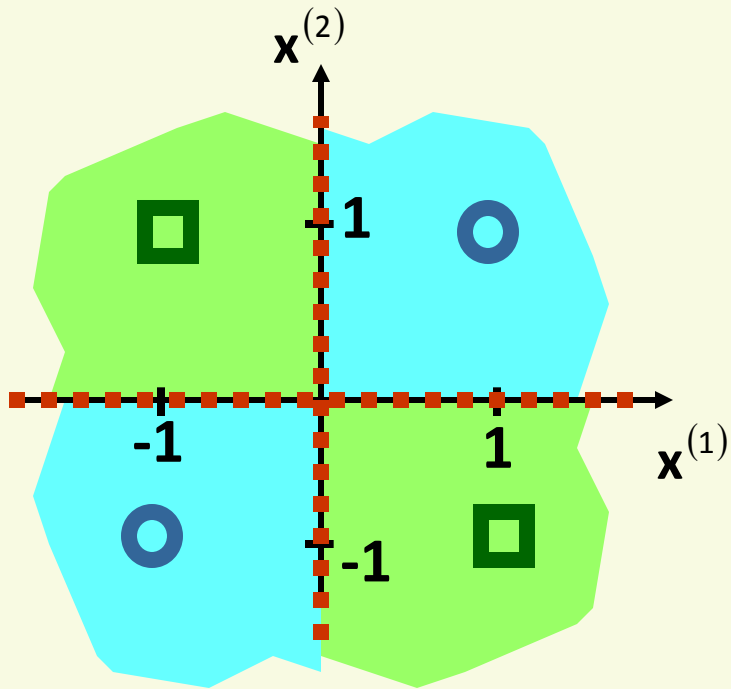
- by plugging in $\mathbf{x}_1 = [1, -1]$, $\mathbf{x}_2 = [-1, 1]$, $\mathbf{x}_3 = [1, 1]$, $\mathbf{x}_4 = [-1, -1]$

- Nonlinear discriminant function is

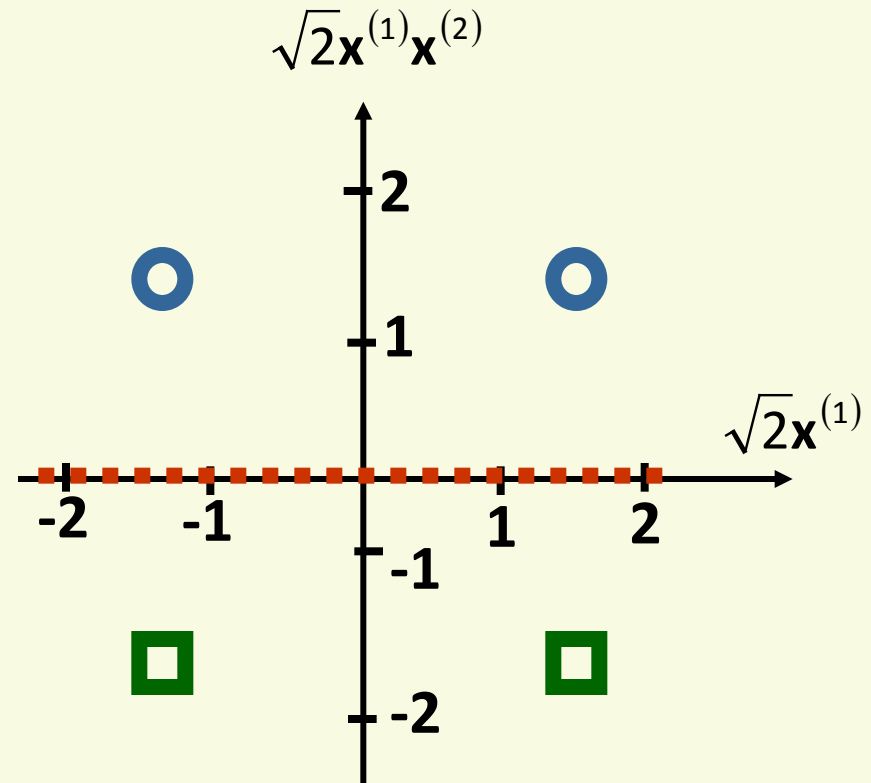
$$\mathbf{g}(\mathbf{x}) = \mathbf{w} \phi(\mathbf{x}) = \sum_{i=1}^6 \mathbf{w}_i \phi_i(\mathbf{x}) = \sqrt{2}(\sqrt{2}\mathbf{x}^{(1)}\mathbf{x}^{(2)}) = 2\mathbf{x}^{(1)}\mathbf{x}^{(2)}$$

SVM Example: XOR Problem

$$g(\mathbf{x}) = -2\mathbf{x}^{(1)}\mathbf{x}^{(2)}$$

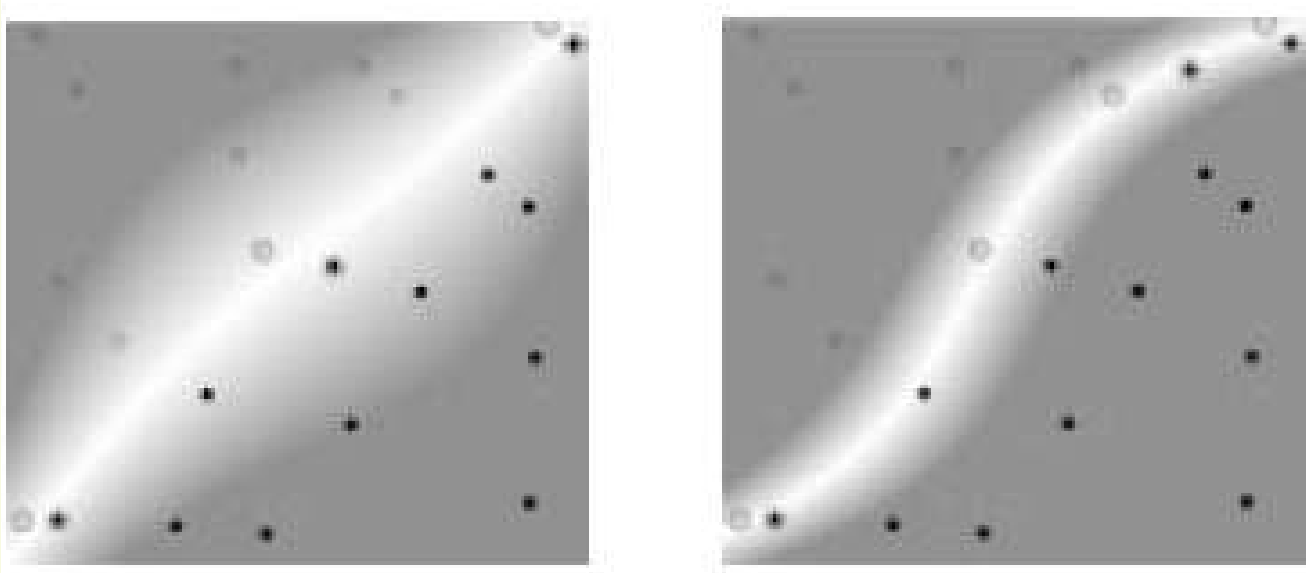


decision boundaries nonlinear



decision boundary is linear

Degree 3 Polynomial Kernel



- Left: In linearly separable case, decision boundary is roughly linear, indicating that dimensionality is controlled
- Right: nonseparable case is handled by a polynomial of degree 3

SVM as Unconstrained Minimization

- SVM formulated as constrained optimization, minimize

$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n \xi_i$$

- constrained to
$$\begin{cases} \mathbf{z}_i (\mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0) \geq 1 - \xi_i & \forall i \\ \xi_i \geq 0 & \forall i \end{cases}$$

- Let us name $\mathbf{f}(\mathbf{x}_i) = \mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0$

- The constraint can be rewritten as
$$\begin{cases} \mathbf{z}_i \mathbf{f}(\mathbf{x}_i) \geq 1 - \xi_i & \forall i \\ \xi_i \geq 0 & \forall i \end{cases}$$

- Which implies $\xi_i = \max(0, 1 - \mathbf{z}_i \mathbf{f}(\mathbf{x}_i))$

- SVM objective can be rewritten as unconstrained optimization

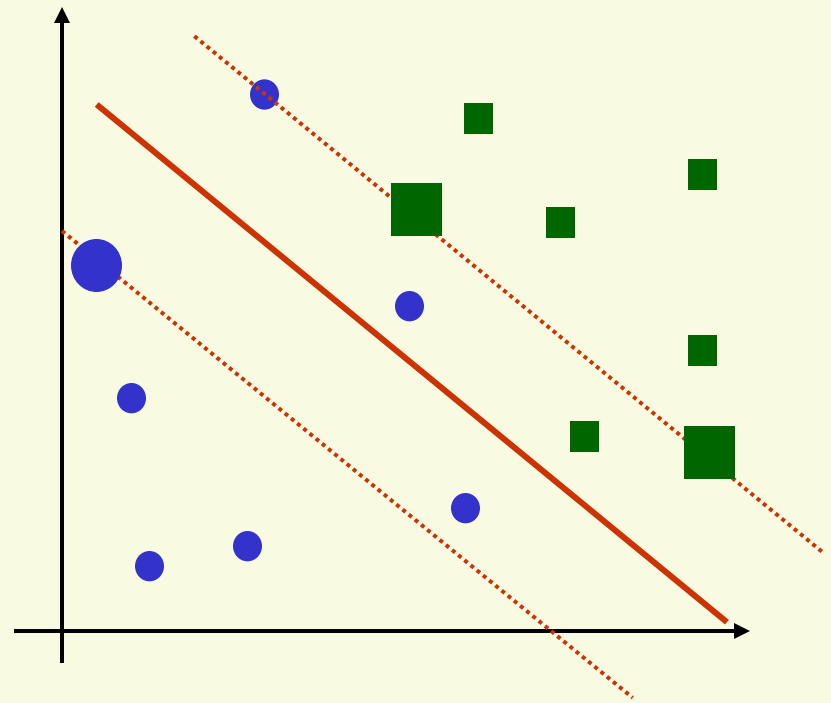
$$J(\mathbf{w}, \xi_1, \dots, \xi_n) = \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{weights regularization}} + \beta \underbrace{\sum_{i=1}^n \max(0, 1 - \mathbf{z}_i \mathbf{f}(\mathbf{x}_i))}_{\text{loss function}}$$

SVM as Unconstrained Minimization

- SVM objective can be rewritten as unconstrained optimization

$$J(\mathbf{w}) = \underbrace{\frac{1}{2} \|\mathbf{w}\|^2}_{\text{weights regularization}} + \beta \underbrace{\sum_{i=1}^n \max(0, 1 - z_i f(\mathbf{x}_i))}_{\text{loss function}}$$

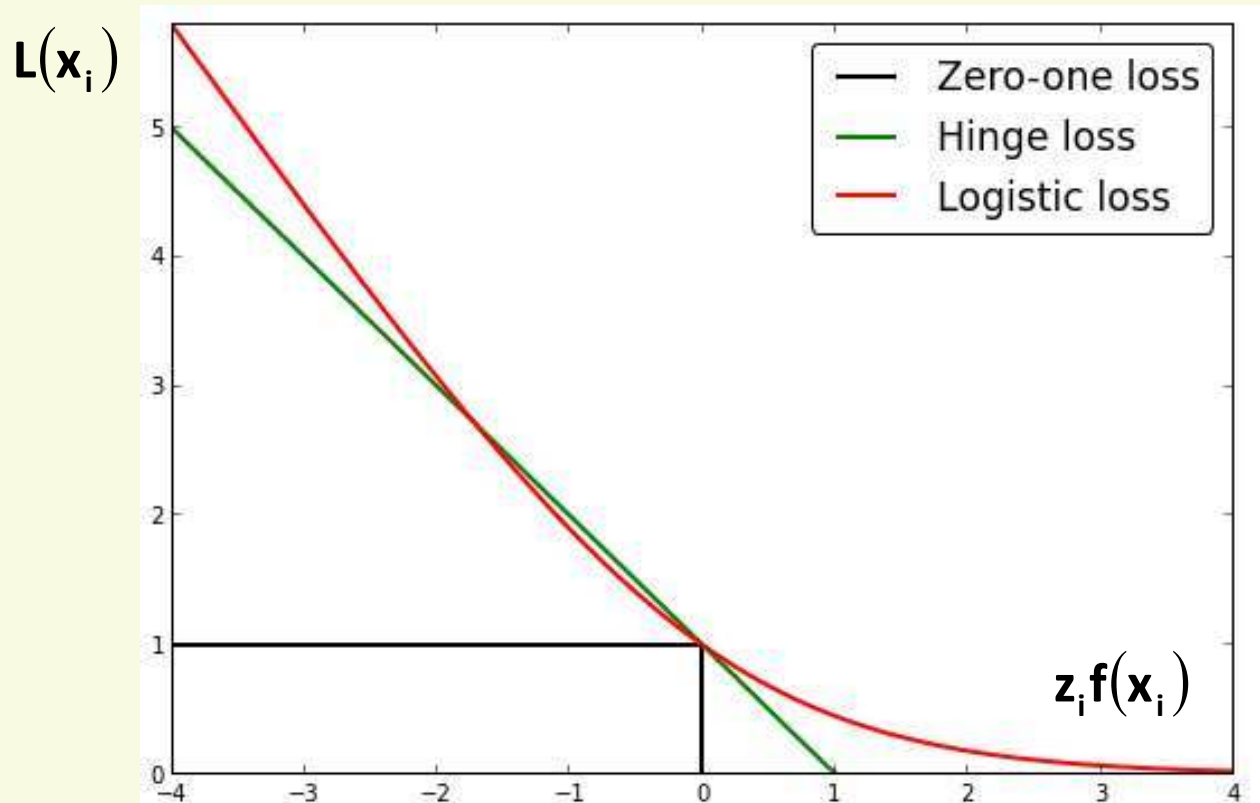
- $z_i f(\mathbf{x}_i) > 1$: $\mathbf{x}_i$ is on the right side of the hyperplane and outside margin, no loss
- $z_i f(\mathbf{x}_i) = 1$: $\mathbf{x}_i$ on the margin, no loss
- $z_i f(\mathbf{x}_i) < 1$: $\mathbf{x}_i$ is inside margin, or on the wrong side of the hyperplane, contributes to loss



SVM: Hinge Loss

- SVM uses Hinge loss per sample $\mathbf{x}_i$

$$L_i(\mathbf{x}_i) = \max(0, 1 - z_i f(\mathbf{x}_i))$$



- Hinge loss encourages classification with a margin of 1

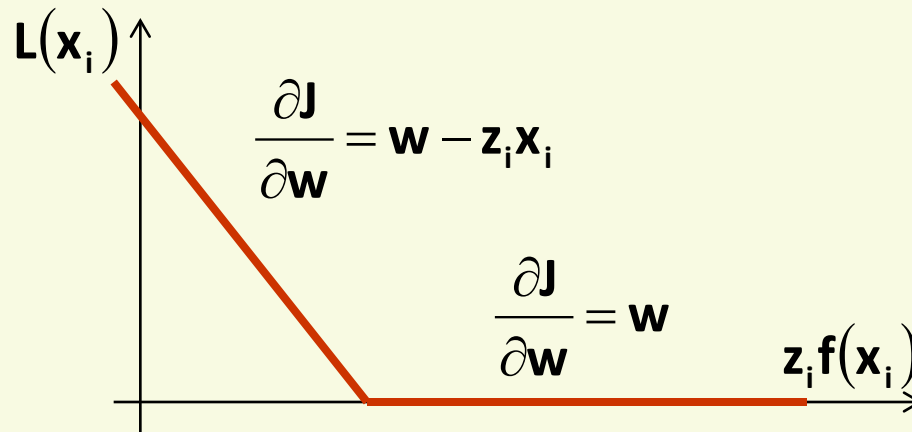
SVM: Hinge Loss

- Can optimize with gradient descent, convex function

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2 + \beta \sum_{i=1}^n \max(0, 1 - \mathbf{z}_i \mathbf{f}(\mathbf{x}_i))$$

$$\mathbf{f}(\mathbf{x}_i) = \mathbf{w}^t \mathbf{x}_i + \mathbf{w}_0$$

- Gradient



- Gradient descent, single sample

$$\mathbf{w} = \begin{cases} \mathbf{w} - \alpha(\mathbf{w} - \beta \mathbf{z}_i \mathbf{x}_i) & \text{if } \mathbf{z}_i \mathbf{f}(\mathbf{x}_i) < 1 \\ \mathbf{w} - \alpha \mathbf{w} & \text{otherwise} \end{cases}$$

SVM Summary

- Advantages:
 - nice theory
 - good generalization properties
 - objective function has no local minima
 - can be used to find non linear discriminant functions
 - often works well in practice, even if not a lot of training data
- Disadvantages:
 - tends to be slower than other methods
 - quadratic programming is computationally expensive